

# Pretentious Self-Parsing and Self-Evaluation of $\lambda$ -Calculus

Heewon Lee

Quantum Many Body Theory Group

October 15, 2026

# Motivation

- Reading strings of  $\lambda$ -expressions and evaluating them is itself a computational task.

# Motivation

- Reading strings of  $\lambda$ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a  $\lambda$ -expression.

# Motivation

- Reading strings of  $\lambda$ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a  $\lambda$ -expression.
- $\Rightarrow$  Lambda evaluation should also be possible with a  $\lambda$ -expression!

# Motivation

- Reading strings of  $\lambda$ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a  $\lambda$ -expression.
- $\Rightarrow$  Lambda evaluation should also be possible with a  $\lambda$ -expression!

$\therefore$  Goal: Parse and evaluate  $\lambda$ -expressions using  $\lambda$ -calculus!

# Concrete and Abstract Syntax

**Concrete Syntax**

**Abstract Syntax**

# Concrete and Abstract Syntax

## Concrete Syntax

$\langle \text{term} \rangle ::= \langle \text{nat} \rangle$  (Variable)  
           $| (\lambda. \langle \text{term} \rangle)$  (Function)  
           $| ((\langle \text{term} \rangle) (\langle \text{term} \rangle))$  (Application)

## Abstract Syntax

# Concrete and Abstract Syntax

## Concrete Syntax

|                                                                         |               |
|-------------------------------------------------------------------------|---------------|
| $\langle \text{term} \rangle ::= \langle \text{nat} \rangle$            | (Variable)    |
| $\quad   (\lambda. \langle \text{term} \rangle)$                        | (Function)    |
| $\quad   ((\langle \text{term} \rangle) (\langle \text{term} \rangle))$ | (Application) |

## Abstract Syntax

|                            |               |
|----------------------------|---------------|
| $t ::= \text{Var}(n)$      | (Variable)    |
| $\quad   \text{Lam}(t)$    | (Function)    |
| $\quad   \text{App}(t, t)$ | (Application) |

# Concrete and Abstract Syntax

## Concrete Syntax

|                                                                         |               |
|-------------------------------------------------------------------------|---------------|
| $\langle \text{term} \rangle ::= \langle \text{nat} \rangle$            | (Variable)    |
| $\quad   (\lambda. \langle \text{term} \rangle)$                        | (Function)    |
| $\quad   ((\langle \text{term} \rangle) (\langle \text{term} \rangle))$ | (Application) |

## Abstract Syntax

|                            |               |
|----------------------------|---------------|
| $t ::= \text{Var}(n)$      | (Variable)    |
| $\quad   \text{Lam}(t)$    | (Function)    |
| $\quad   \text{App}(t, t)$ | (Application) |

e.g.) “ $(\lambda. 0)$ ”  $\rightarrow$  [LPAR; LAM; NAT(0); LPAR]  $\rightarrow$   $\lambda$ -repr. of tokens  $\rightarrow$   $\lambda$ -repr. of AST  $\rightarrow$   $\lambda$ -expr.

# Concrete and Abstract Syntax

## Concrete Syntax

|                                                                         |               |
|-------------------------------------------------------------------------|---------------|
| $\langle \text{term} \rangle ::= \langle \text{nat} \rangle$            | (Variable)    |
| $\quad   (\lambda. \langle \text{term} \rangle)$                        | (Function)    |
| $\quad   ((\langle \text{term} \rangle) (\langle \text{term} \rangle))$ | (Application) |

## Abstract Syntax

|                            |               |
|----------------------------|---------------|
| $t ::= \text{Var}(n)$      | (Variable)    |
| $\quad   \text{Lam}(t)$    | (Function)    |
| $\quad   \text{App}(t, t)$ | (Application) |

e.g.) “ $(\lambda. 0)$ ”  $\rightarrow$  [LPAR; LAM; NAT(0); LPAR]  $\rightarrow$   $\lambda$ -repr. of tokens  $\rightarrow$   $\lambda$ -repr. of AST  $\rightarrow$   $\lambda$ -expr.

- 1 Represent each token
- 2 Represent each AST constructor
- 3 Parser `e_parse` from token list to AST
- 4 Evaluator `e_eval` from AST to  $\lambda$ -expression

# Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types

# Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

# Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

| Constructor            | Scott Encoding                 |
|------------------------|--------------------------------|
| $\text{Var}(n)$        | $\lambda v / a. v \ n$         |
| $\text{Lam}(t)$        | $\lambda v / a. \lambda t$     |
| $\text{App}(t_1, t_2)$ | $\lambda v / a. a \ t_1 \ t_2$ |
| <hr/>                  |                                |
| LPAR                   | $\lambda / r \ a \ v. l$       |
| RPAR                   | $\lambda / r \ a \ v. r$       |
| LAM                    | $\lambda / r \ a \ v. a$       |
| $\text{NAT}(n)$        | $\lambda / r \ a \ v. v \ n$   |

# Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

| Constructor | Scott Encoding |
|-------------|----------------|
|-------------|----------------|

|                 |                        |
|-----------------|------------------------|
| $\text{Var}(n)$ | $\lambda v / a. v \ n$ |
|-----------------|------------------------|

|                 |                        |
|-----------------|------------------------|
| $\text{Lam}(t)$ | $\lambda v / a. / \ t$ |
|-----------------|------------------------|

|                        |                                |
|------------------------|--------------------------------|
| $\text{App}(t_1, t_2)$ | $\lambda v / a. a \ t_1 \ t_2$ |
|------------------------|--------------------------------|

---

|      |                          |
|------|--------------------------|
| LPAR | $\lambda / r \ a \ v. /$ |
|------|--------------------------|

|      |                          |
|------|--------------------------|
| RPAR | $\lambda / r \ a \ v. r$ |
|------|--------------------------|

|     |                          |
|-----|--------------------------|
| LAM | $\lambda / r \ a \ v. a$ |
|-----|--------------------------|

|            |                              |
|------------|------------------------------|
| NAT( $n$ ) | $\lambda / r \ a \ v. v \ n$ |
|------------|------------------------------|

## Case statement

$$t \ fv \ fl \ fa = \begin{cases} fv \ n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

# Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

| Constructor | Scott Encoding |
|-------------|----------------|
|-------------|----------------|

|            |                        |
|------------|------------------------|
| Var( $n$ ) | $\lambda v / a. v \ n$ |
|------------|------------------------|

|            |                        |
|------------|------------------------|
| Lam( $t$ ) | $\lambda v / a. l \ t$ |
|------------|------------------------|

|                   |                                |
|-------------------|--------------------------------|
| App( $t_1, t_2$ ) | $\lambda v / a. a \ t_1 \ t_2$ |
|-------------------|--------------------------------|

---

|      |                          |
|------|--------------------------|
| LPAR | $\lambda / r \ a \ v. l$ |
|------|--------------------------|

|      |                          |
|------|--------------------------|
| RPAR | $\lambda / r \ a \ v. r$ |
|------|--------------------------|

|     |                          |
|-----|--------------------------|
| LAM | $\lambda / r \ a \ v. a$ |
|-----|--------------------------|

|            |                              |
|------------|------------------------------|
| NAT( $n$ ) | $\lambda / r \ a \ v. v \ n$ |
|------------|------------------------------|

## Case statement

$$t \text{ fv } fl \text{ fa} = \begin{cases} \text{fv } n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Example:  $\lambda x. \lambda y. x$

$\lambda v / a. l$

# Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

| Constructor | Scott Encoding |
|-------------|----------------|
|-------------|----------------|

|            |                        |
|------------|------------------------|
| Var( $n$ ) | $\lambda v / a. v \ n$ |
|------------|------------------------|

|            |                        |
|------------|------------------------|
| Lam( $t$ ) | $\lambda v / a. / \ t$ |
|------------|------------------------|

|                   |                                |
|-------------------|--------------------------------|
| App( $t_1, t_2$ ) | $\lambda v / a. a \ t_1 \ t_2$ |
|-------------------|--------------------------------|

|      |                          |
|------|--------------------------|
| LPAR | $\lambda / r \ a \ v. /$ |
|------|--------------------------|

|      |                          |
|------|--------------------------|
| RPAR | $\lambda / r \ a \ v. r$ |
|------|--------------------------|

|     |                          |
|-----|--------------------------|
| LAM | $\lambda / r \ a \ v. a$ |
|-----|--------------------------|

|            |                              |
|------------|------------------------------|
| NAT( $n$ ) | $\lambda / r \ a \ v. v \ n$ |
|------------|------------------------------|

## Case statement

$$t \ fv \ fl \ fa = \begin{cases} fv \ n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Example:  $\lambda x. \lambda y. x$

$$\lambda v / a. / (\lambda v / a. /$$

# Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

| Constructor | Scott Encoding |
|-------------|----------------|
|-------------|----------------|

|            |                        |
|------------|------------------------|
| Var( $n$ ) | $\lambda v / a. v \ n$ |
|------------|------------------------|

|            |                            |
|------------|----------------------------|
| Lam( $t$ ) | $\lambda v / a. \lambda t$ |
|------------|----------------------------|

|                   |                                |
|-------------------|--------------------------------|
| App( $t_1, t_2$ ) | $\lambda v / a. a \ t_1 \ t_2$ |
|-------------------|--------------------------------|

|      |                                |
|------|--------------------------------|
| LPAR | $\lambda / r \ a \ v. \lambda$ |
|------|--------------------------------|

|      |                          |
|------|--------------------------|
| RPAR | $\lambda / r \ a \ v. r$ |
|------|--------------------------|

|     |                          |
|-----|--------------------------|
| LAM | $\lambda / r \ a \ v. a$ |
|-----|--------------------------|

|            |                              |
|------------|------------------------------|
| NAT( $n$ ) | $\lambda / r \ a \ v. v \ n$ |
|------------|------------------------------|

## Case statement

$$t \text{ fv } fl \text{ fa} = \begin{cases} \text{fv } n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Example:  $\lambda x. \lambda y. x$

$$\lambda v / a. \lambda (\lambda v / a. \lambda (\lambda v / a. a \quad ))$$

# Theory: Algebraic Data Types and Scott Encoding

- Algebraic data type (ADT, 조립식 타입): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

| Constructor | Scott Encoding |
|-------------|----------------|
|-------------|----------------|

|            |                        |
|------------|------------------------|
| Var( $n$ ) | $\lambda v / a. v \ n$ |
|------------|------------------------|

|            |                            |
|------------|----------------------------|
| Lam( $t$ ) | $\lambda v / a. \lambda t$ |
|------------|----------------------------|

|                   |                                |
|-------------------|--------------------------------|
| App( $t_1, t_2$ ) | $\lambda v / a. a \ t_1 \ t_2$ |
|-------------------|--------------------------------|

---

|      |                                |
|------|--------------------------------|
| LPAR | $\lambda / r \ a \ v. \lambda$ |
|------|--------------------------------|

|      |                          |
|------|--------------------------|
| RPAR | $\lambda / r \ a \ v. r$ |
|------|--------------------------|

|     |                          |
|-----|--------------------------|
| LAM | $\lambda / r \ a \ v. a$ |
|-----|--------------------------|

|            |                              |
|------------|------------------------------|
| NAT( $n$ ) | $\lambda / r \ a \ v. v \ n$ |
|------------|------------------------------|

## Case statement

$$t \text{ fv } fl \text{ fa} = \begin{cases} \text{fv } n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Example:  $\lambda x. \lambda y. x$

$$\lambda v / a. \lambda (\lambda v / a. \lambda (\lambda v / a. a (\lambda s z. s \ z)))$$

# Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$  Scott encoding: match expressions

# Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$  Scott encoding: match expressions

| Constructor | Church             | Scott                             |
|-------------|--------------------|-----------------------------------|
| 0           | $\lambda s z. z$   | $\lambda s z. z$                  |
| succ $n$    | $\lambda s z. s z$ | $\lambda s z. s (\lambda s z. z)$ |

---

# Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$  Scott encoding: match expressions

| Constructor       | Church                                     | Scott                             |
|-------------------|--------------------------------------------|-----------------------------------|
| 0                 | $\lambda s z. z$                           | $\lambda s z. z$                  |
| succ $n$          | $\lambda s z. s z$                         | $\lambda s z. s (\lambda s z. z)$ |
| Var( $n$ )        | $\lambda v / a. v n$                       | $\lambda v / a. v n$              |
| Lam( $t$ )        | $\lambda v / a. \lambda t v / a$           | $\lambda v / a. \lambda t$        |
| App( $t_1, t_2$ ) | $\lambda v / a. a (t_1 v / a) (t_2 v / a)$ | $\lambda v / a. a t_1 t_2$        |

# Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$  Scott encoding: match expressions

| Constructor       | Church                                                 | Scott                             |
|-------------------|--------------------------------------------------------|-----------------------------------|
| 0                 | $\lambda s z. z$                                       | $\lambda s z. z$                  |
| succ $n$          | $\lambda s z. s z$                                     | $\lambda s z. s (\lambda s z. z)$ |
| Var( $n$ )        | $\lambda v / a. v n$                                   | $\lambda v / a. v n$              |
| Lam( $t$ )        | $\lambda v / a. \lambda (t \ v \ / \ a)$               | $\lambda v / a. \lambda t$        |
| App( $t_1, t_2$ ) | $\lambda v / a. a (t_1 \ v \ / \ a) (t_2 \ v \ / \ a)$ | $\lambda v / a. a \ t_1 \ t_2$    |

- Church: Top-level  $v, l, a$  are passed to every subexpression (Bottom-up)
- Scott: Top-level  $v, l, a$  operate on the immediate subexpressions directly (Top-down)

# Theory: Church vs. Scott Encoding

- General Church encoding for ADTs: Consume with fold operations
- $\leftrightarrow$  Scott encoding: match expressions

| Constructor       | Church                                     | Scott                             |
|-------------------|--------------------------------------------|-----------------------------------|
| 0                 | $\lambda s z. z$                           | $\lambda s z. z$                  |
| succ $n$          | $\lambda s z. s z$                         | $\lambda s z. s (\lambda s z. z)$ |
| Var( $n$ )        | $\lambda v / a. v n$                       | $\lambda v / a. v n$              |
| Lam( $t$ )        | $\lambda v / a. \lambda t v / a$           | $\lambda v / a. \lambda t$        |
| App( $t_1, t_2$ ) | $\lambda v / a. a (t_1 v / a) (t_2 v / a)$ | $\lambda v / a. a t_1 t_2$        |

- Church: Top-level  $v, l, a$  are passed to every subexpression (Bottom-up)
- Scott: Top-level  $v, l, a$  operate on the immediate subexpressions directly (Top-down)
- For parsing and evaluating, **Scott encoding** is more suitable!

# Self-Interpreter e\_eval

- Suppose  $\lceil M \rceil$  is the AST encoding of a  $\lambda$ -expression  $M$ .

# Self-Interpreter e\_eval

- Suppose  $\ulcorner M \urcorner$  is the AST encoding of a  $\lambda$ -expression  $M$ .
- Interpreter `e_eval` must satisfy  $\text{App}(\text{e\_eval}, \ulcorner M \urcorner) \simeq_{\beta} M$ .

## Self-Interpreter e\_eval

- Suppose  $\ulcorner M \urcorner$  is the AST encoding of a  $\lambda$ -expression  $M$ .
- Interpreter `e_eval` must satisfy  $\text{App}(\text{e\_eval}, \ulcorner M \urcorner) \simeq_{\beta} M$ .

`e_eval =  $\lambda ast.$  eval_comb ast nil`      (List of variables that have been defined)

# Self-Interpreter e\_eval

- Suppose  $\ulcorner M \urcorner$  is the AST encoding of a  $\lambda$ -expression  $M$ .
- Interpreter e\_eval must satisfy  $\text{App}(\text{e\_eval}, \ulcorner M \urcorner) \simeq_{\beta} M$ .

$\text{e\_eval} = \lambda ast. \text{eval\_comb } ast \text{ nil}$       (List of variables that have been defined)

$\text{eval\_comb} = \lambda ast \text{ lst. } ast$

$(\lambda n. \text{lookup } lst \ n)$

(Var: Lookup)

$(\lambda t. \text{eval\_comb } t \ (\text{cons } lst \ (\text{Var } 0)))$

(Lam: Define variable, evaluate)

$(\lambda t_1 \ t_2. (\text{eval\_comb } t_1 \ lst) \ (\text{eval\_comb } t_2 \ lst))$

(App: Evaluate each, apply)

# Self-Interpreter e\_eval

- Suppose  $\ulcorner M \urcorner$  is the AST encoding of a  $\lambda$ -expression  $M$ .
- Interpreter `e_eval` must satisfy  $\text{App}(\text{e\_eval}, \ulcorner M \urcorner) \simeq_{\beta} M$ .

`e_eval` =  $\lambda ast. \text{eval\_comb } ast \text{ nil}$       (List of variables that have been defined)

`eval_comb` =  $\lambda ast \text{ lst}. ast$

$(\lambda n. \text{lookup } lst \ n)$

(Var: Lookup)

$(\lambda t. \text{eval\_comb } t \ (\text{cons } lst \ (\text{Var } 0)))$

(Lam: Define variable, evaluate)

$(\lambda t_1 \ t_2. (\text{eval\_comb } t_1 \ lst) \ (\text{eval\_comb } t_2 \ lst))$

(App: Evaluate each, apply)

- `cons`: Push every index up by one
- Recursion via  $Y$ -combinator